

머신러닝 I

02 머신러닝을 준비하기 위한 라이브러리

목차 02 머신러닝을 준비하기 위한 라이브러리

1. 배열 데이터 처리를 위한 라이브러리 – NumPy
2. 배열 데이터 처리를 위한 라이브러리 – Pandas
3. 데이터 시각화를 위한 라이브러리 – Matplotlib
4. 데이터 시각화를 위한 라이브러리 – Seaborn

1. 배열 데이터 처리를 위한 라이브러리 - NumPy



같은 종류의 데이터를 순차적으로 저장하는 자료 구조

- 배열의 모든 요소는 동일한 데이터 타입을 가짐
- 효율적인 데이터 관리
- 메모리에 연속적으로 저장
 - 인덱스를 통해 빠르게 접근 가능

```
a = [1, 3, 5, 3, 15]    #1차원 벡터
2*a # [1, 3, 5, 3, 15, 1, 3, 5, 3, 15]
a+a # [1, 3, 5, 3, 15, 1, 3, 5, 3, 15]
```

- 파이썬에서 배열은 주로 **리스트(List)**로 구현
- **리스트의 연산 결과 ≠ 벡터의 연산**

```
import array as ar
a = ar.array('i', [1, 2, 3])  # 'i'는 정수형 데이터를 의미
b = ar.array('i', [4, 5, 6])
print(a+b)  # array('i', [1, 2, 3, 4, 5, 6])
```

- 파이썬의 리스트는 숫자 뿐 아니라 **다양한 데이터형** 사용 가능
- Array 객체에 **사칙 연산** 적용 시, **리스트형과 동일하게 처리**
→ **벡터와 행렬**을 처리하기에 **적합하지 않음**

과학 연산에 사용되는 다차원 배열 처리를 위한 패키지

- 다차원 배열 지원
- 빠르고 효율적인 연산
- 다양한 수학 함수 제공



```
# import 할 때는 주로 `np`를 사용  
import numpy as np
```

- NumPy에 존재하는 다양한 함수들을 이용할 수 있음
- 대표적인 내장 함수
 - array()
 - arange()



```
a = np.array([0, 1, 2, 3])  
# array([0, 1, 2, 3])  
  
b = np.array([0, 1, 2, 3], dtype=float)  
# array([0., 1., 2., 3.] )
```

- 파이썬의 **리스트**를 **numpy 배열의 자료형으로 변환**하는 함수
- 다양한 데이터 타입을 지원하며, **데이터 타입을 명시적으로 지정**할 수 있음



```
a = np.arange(10) # 범위 (0~9) 인자가 1개일 때  
b = np.arange(1,4) # 범위(1~3), 인자가 2개일 때  
c = np.arange(1,8,2) # 범위(1~8), 2씩 증가 (1,3,5,7)
```

- **arange(start, stop, step)**
 - start부터 stop 전까지 step 만큼의 크기로 증가하는 배열을 선언



```
zero_arr1 = np.zeros(5)      # [0., 0., 0., 0., 0.]  
zero_arr2 = np.zeros(5, dtype=int)  # [0, 0, 0, 0, 0]
```

- **zeros(shape, dtype)**

- 선언하고 싶은 배열의 크기를 입력하여 **0으로 채워진 배열**을 선언
- 기본 데이터 타입은 float



```
one_arr1 = np.ones(5)      # [1., 1., 1., 1., 1.]  
one_arr2 = np.ones(5, dtype=int)  # [1, 1, 1, 1, 1]
```

- **ones(shape, dtype)**
 - 선언하고 싶은 배열의 크기를 입력하여 **1으로 채워진 배열**을 선언
 - 기본 데이터 타입은 float



```
random_arr1 = np.random.rand(5)
# [0.12345678 0.23456789 0.34567890 0.45678901 0.56789012]
random_arr2 = np.random.rand(2,3)
# [[0.35609002, 0.97503756, 0.61344139], [0.22103651, 0.8687285 , 0.34845594]]
```

- **random.rand(d0, d1, ..., dn)**
 - 0과 1사이의 균일 분포에서 **난수를 생성**하여 배열을 반환하는 함수
 - 배열의 각 차원의 크기를 나타내는 정수값 전달



```
random_arr2 = np.random.randint(0, 10, size=5)    #lower는 0, high는 10, size는 5  
random_arr2    # [2, 8, 4, 3, 9]
```

- **random.randint(low, high=None, size=None)**
 - 주어진 범위 내에서 **정수 난수를 생성**하여 배열을 반환하는 함수
 - low: 생성할 난수의 최솟값 (포함)
 - high: 생성할 난수의 최댓값 (포함 X)



	[0]	[1]	[2]	[3]
[0]	[0,0]	[0,1]	[0,2]	[0,3]
[1]	[1,0]	[1,1]	[1,2]	[1,3]
[2]	[2,0]	[2,1]	[2,2]	[2,3]

배열의 인덱스는 0부터 시작



```
c = np.array([[1,2,3],[4,5,6]])  
c.shape      #(2, 3)  
c.size       #6
```

- **.shape**
 - 배열의 **row와 column의 크기**를 출력
- **.size**
 - 배열 **성분의 총 개수**를 출력



```
c = np.array([[1,2,3], [4,5,6]])  
c.dtype      # dtype('int64')  
c[1,2]       # 6
```

- **.dtype**
 - 배열 성분의 데이터형을 알 수 있음
- numpy 배열 **[] 안에 index 값**을 넣으면 해당 성분을 출력



```
a = np.array([1,2,3,4])  
a = np.reshape(a, (2, 2)) # 원래 a의 size와 변경할 a의 size는 같아야 함  
  
a = a.reshape((2,2)) # 이와 같은 방법으로도 사용 가능  
# [[1, 2], [3,4]]
```

- **reshape(arr, newshape)**
 - 배열의 형태를 입력하여 **배열의 구조를 변경**해주는 함수



```
# 인덱스 정보를 활용하여 자르기
a0 = np.array([[1,2,3],[4,5,6]]) # (2,3) 크기의 행렬 a0 선언
a1 = a0[:,0] # column 벡터로 자르기
a2 = a0[:, -1] # 마지막 column 벡터로 자르기
a3 = a0[1,:] # row 벡터로 자르기
a4 = a0[-1,:] # 마지막 row 벡터로 자르기
```

```
a1 = [1, 4]
a2 = [3, 6]
a3 = [4, 5, 6]
a4 = [4, 5, 6]
```

- 배열의 **인덱스 정보**를 활용하여 NumPy 배열을 자를 수 있음



```
a0 = np.array([[1,2,3,4,5,6],[7,8,9,10,11,12]]) # (2,6) 크기의 행렬 a 선언
a1 = a0[:,4:] # `배열[시작값:]`은 `시작값` 인덱스를 시작으로 `배열의 마지막 인덱스`까지
의 배열을 표시
a2 = a0[:,3:6] # `도착값`은 마지막 인덱스 값보다 1이 크므로, column 벡터의 크기가 6인 a
배열의 `도착값`으로 6을 입력하면 5번 인덱스를 의미
a3 = a0[:,2:-1] # `-1`은 마지막 인덱스를 의미하기에 4번 인덱스까지를 출력
a4 = a0[:,0:3:2] # 간격이 2이기에 0번, 2번 인덱스만을 표시
```

```
a1 = [[5, 6], [11, 12]]
a2 = [[4, 5, 6], [10, 11, 12]]
a3 = [[3, 4, 5], [9, 10, 11]]
a4 = [[1, 3], [7, 9]]
```

- 배열의 **시작값**부터 **도착값** 사이의 값을 **간격**을 두고 자를 수 있음



```
a = np.array([[1,2,3],[4,5,6]]) # (2,3) 크기의 행렬 a 선언  
b = np.array([[7,8,9],[10,11,12]]) # (2,3) 크기의 행렬 b 선언  
np.r_[a,b] # row 를 추가
```

```
[[ 1,  2,  3],  
 [ 4,  5,  6],  
 [ 7,  8,  9],  
 [10, 11, 12]]
```

- **r_[a, b]**
 - **행 방향**으로 배열을 연결
 - 1차원 배열끼리 결합 시, **2차원 배열**로 만듦
 - 2차원 배열인 경우, 동일한 축을 따라 결합



```
a = np.array([[1,2,3],[4,5,6]]) # (2,3) 크기의 행렬 a 선언  
b = np.array([[7,8,9],[10,11,12]]) # (2,3) 크기의 행렬 b 선언  
np.c_[a,b] #column을 추가
```

```
[[ 1,  2,  3,  7,  8,  9],  
 [ 4,  5,  6, 10, 11, 12]]
```

- **c_[a, b]**
 - **열 방향**으로 배열을 연결
 - 배열 연결에 주로 사용되지만, 슬라이싱을 활용한 배열 생성도 가능



```
a = np.array([[1,2,3],[4,5,6]]) # (2,3) 크기의 행렬 a 선언  
b = np.array([[7,8,9],[10,11,12]]) # (2,3) 크기의 행렬 b 선언  
np.concatenate((a,b), axis=0) # 0은 row 를 의미함
```

```
[[ 1,  2,  3],  
 [ 4,  5,  6],  
 [ 7,  8,  9],  
 [10, 11, 12]]
```

- **concatenate()**

- 두 개 이상의 배열을 **축을 따라 연결**하여 새로운 배열 생성
- 연결되는 축을 제외한 나머지 차원의 크기가 동일해야 함



배열 덧셈

```
a = np.array([[1, 2, 3], [3, 2, 5]])
```

```
b = np.array([[-1, 3, 5], [1, 4, 2]])
```

a+b

배열 뺄셈

a-b

배열 *, / 연산은 같은 인덱스에 존재하는 성분끼리 곱하거나 나눈 형태를 출력

a*b

a/b # 소수점 여덟번째자리까지 출력

a+b

```
[[0, 5, 8],
```

```
 [4, 6, 7]]
```

a-b

```
[[ 2, -1, -2],
```

```
 [ 2, -2,  3]]
```

a*b

```
[[ -1,  6, 15],
```

```
 [ 3,  8, 10]]
```

a/b

```
[[ -1.          ,  0.66666667,  0.6          ],
```

```
 [ 3.          ,  0.5          ,  2.5          ]]
```

- numpy 배열의 연산은 '+, -, *, /' 기호로 간단하게 수행 가능



- 다음과 같은 2x3 행렬 A, B를 계산하는 경우는?

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 2 & 5 \end{bmatrix} \quad B = \begin{bmatrix} -1 & 3 & 5 \\ 1 & 4 & 2 \end{bmatrix}$$

$$A + B = \begin{bmatrix} 1 - 1 & 2 + 3 & 3 + 5 \\ 3 + 1 & 2 + 4 & 5 + 2 \end{bmatrix} = \begin{bmatrix} 0 & 5 & 8 \\ 4 & 6 & 7 \end{bmatrix}$$

$$A - B = \begin{bmatrix} 1 - (-1) & 2 - 3 & 3 - 5 \\ 3 - 1 & 2 - 4 & 5 - 2 \end{bmatrix} = \begin{bmatrix} 2 & -1 & -2 \\ 2 & -2 & 3 \end{bmatrix}$$

2. 배열 데이터 처리를 위한 라이브러리 - Pandas

Pandas는 **row와 colum**으로 이루어진 데이터 객체를 생성, 수정하기 위한 패키지

- **데이터 분석**에 초점이 맞춰져 빅데이터를 다루는데 편리
 - numpy는 수학적 계산을 위한 배열에 초점



Pandas 라이브러리 import 하기

머신러닝 I

02 머신러닝을 준비하기
위한 라이브러리

2. 배열 데이터 처리를 위
한 라이브러리 -
Pandas

```
# import 할 때는 주로 `pd`를 사용  
import pandas as pd
```

- Pandas에 존재하는 다양한 함수들을 이용할 수 있음
- 데이터 구조와 데이터 조작 기능 제공

1차원 배열과 같은 자료 구조

1차원 배열의 값 뿐 아니라 각 값에 연결된 인덱스 값 저장

	apples
0	3
1	2
2	0
3	1

```
# Series 정의하기  
obj = pd.Series([4, 7, -5])
```

```
0      4  
1      7  
2     -5  
dtype: int64
```

- 0부터 시작하는 **인덱스와 숫자**가 저장됨



Series 정의하기

```
obj = pd.Series([4, 7, -5 ], index = ['2016-01-01', '2016-01-02', '2016-01-03'])
```

2016-01-01	4
2016-01-02	7
2016-01-03	-5

dtype: int64

- 인덱싱 값을 사용자가 설정할 수 있음
- Index에 인덱싱할 값을 넘겨주면 해당 값으로 인덱싱



```
stock0 = pd.Series([10, 20, 30], index=['naver', 'skt', 'LG'])
stock1 = pd.Series([30, 20, 10], index=['LG', 'naver', 'skt'])
merge = stock0 + stock1
```

```
LG      60
naver   30
skt     30
dtype: int64
```

- 인덱싱의 순서가 서로 다른 경우에도, **인덱싱이 같은 값끼리 덧셈**을 수행



여러 개의 Series가 모여서 행과 열을 이룬 데이터

	apples	orange	bananas
0	3	5	1
1	2	1	5
2	0	3	4
3	1	4	3



```
raw_data = {'col0': [1, 2, 3, 4],  
            'col1': [10, 20, 30, 40],  
            'col2': [100, 200, 300, 400]}  
data = DataFrame(raw_data)
```

- DataFrame 객체를 생성하기 위해서는 **딕셔너리**를 사용
- **각 컬럼에 저장된 데이터를 사용하여 DataFrame 객체로 생성**



	col0	col1	col2
0	1	10	100
1	2	20	200
2	3	30	300
3	4	40	400

- 'col0', 'col1', 'col2'은 DataFrame의 **각 열을 인덱싱** 하는데 사용
- **행 방향**으로는 Series와 유사하게 **정수값으로 자동으로 인덱싱**



```
daeshin = {'open': [11650, 11100, 11200, 11100, 11000],  
          'high': [12100, 11800, 11200, 11100, 11150],  
          'low': [11600, 11050, 10900, 10950, 10900],  
          'close': [11900, 11600, 11000, 11100, 11050]}  
daeshin_day = DataFrame(daeshin, index = [0,2,4,6,8], columns=['open','high','low','close'])
```

- **'col0', 'col1', 'col2'**은 DataFrame의 **각 열을 인덱싱** 하는데 사용
- **행 방향**으로는 Series와 유사하게 **정수값으로 자동으로 인덱싱**



```
a = [[1, 2], [3, 4]]  
df = DataFrame(a, index = [0, 1], columns = ['a', 'b'])
```

	a	b
0	1	2
1	3	4

- **인덱싱에 사용할 값**을 만든 후, DataFrame 객체 생성 시점에 지정



라벨 기반 인덱스를 참조하는 인덱싱/슬라이싱

- **.loc[idx]**
 - DataFrame df에서 **idx 라벨에 해당하는 값**을 추출
- **.loc[idx0 : idx2]**
 - DataFrame df에서 **idx0부터 idx2 라벨에 해당하는 값**들을 추출
 - 슬라이싱 시, **종료 라벨을 포함함**



```
data = {'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]}  
df = pd.DataFrame(data, index=['a', 'b', 'c'])  
print(df.loc['a':'b', ['A', 'C'])
```

	A	C
a	1	7
b	2	8

- loc 함수를 이용하여 행 라벨 a부터 b까지, 열 라벨 A와, C까지 출력한 결과



정수 기반 인덱스를 참조하는 인덱싱

- **.iloc[idx]**
 - DataFrame df에서 **idx** 인덱스에 해당하는 값들을 추출
- **.iloc[idx0 : idx2]**
 - DataFrame df에서 **idx0**부터 **idx2** 직전 인덱스에 해당하는 값들을 추출
 - 슬라이싱 시, **종료 인덱스를 포함하지 않음**



```
data = {'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]}  
df = pd.DataFrame(data, index=['a', 'b', 'c'])  
print(df.iloc[0:2, [0,2]])
```

	A	C
a	1	7
b	2	8

- iloc 함수를 이용하여 행 인덱스 0부터 2 전까지, 열 인덱스 0와, 2까지 출력한 결과



DataFrame에서 행 또는 열을 삭제하기 위한 함수

- **.drop(labels, axis, inplace)**
 - labels = 삭제할 행 또는 열의 이름
 - axis = 삭제할 대상의 축 {0 : 행, 1 : 열}
 - inplace = 원본 객체 수정 여부(default=False)



데이터 삭제 – drop()

머신러닝 I

02 머신러닝을 준비하기
위한 라이브러리

2. 배열 데이터를 처리를 위
한 라이브러리 –
Pandas

```
data = {'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]}  
df = pd.DataFrame(data, index=['a', 'b', 'c'])  
df_dropped = df.drop(labels='a')
```

	A	B	C
b	2	5	8
c	3	6	9

- drop 함수를 이용해 a 행을 삭제한 출력 결과



```
data = {'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]}  
df = pd.DataFrame(data, index=['a', 'b', 'c'])  
df_dropped = df.drop(labels='A', axis=1)
```

	B	C
a	4	7
b	5	8
c	6	9

- drop 함수를 이용해 A 열을 삭제한 출력 결과



데이터프레임 또는 시리즈의 값을 기준으로 정렬하는데 사용

- `.sort_values(by, axis, ascending, inplace)`
 - `by` = 정렬을 수행할 기준이 되는 열 또는 열의 리스트
 - `axis` = 정렬할 축 {0 : 행, 1 : 열}
 - `ascending` = 오름차순 정렬 여부 (default=True)
 - `inplace` = 원본 객체 수정 여부 (default=False)



데이터 정렬 - sort_values()

머신러닝 I

02 머신러닝을 준비하기
위한 라이브러리

2. 배열 데이터 처리를 위
한 라이브러리 -
Pandas

```
data = {'A': [3, 2, 1], 'B': [4, 5, 6]}  
df = pd.DataFrame(data)  
sorted_df = df.sort_values(by='A')
```

	A	B
2	1	6
1	2	5
0	3	4

- A열 기준으로 정렬한 출력 결과
- index 순서 또한 바뀌어진 것을 확인할 수 있음



- `.add(other, axis, fill_value)`
- `.sub(other, axis, fill_value)`
- `.mul(other, axis, fill_value)`
- `.div(other, axis, fill_value)`
 - other = 연산할 데이터프레임 또는 시리즈
 - axis = 수행할 축 지정
 - fill_value = 연산할 데이터프레임이나 시리즈 내 **누락값(NaN)**이 있을 때, 해당 값을 대체 할 값



- **.to_csv(data)**
 - 데이터프레임을 csv 파일로 저장해주는 함수
- **.to_excel(data)**
 - 데이터프레임을 excel 파일로 저장해주는 함수
- **read_csv(data)**
 - csv 파일을 데이터프레임 형식으로 불러오는 함수
- **read_excel(data)**
 - excel 파일을 데이터프레임 형식으로 불러오는 함수
- **data = 저장하거나 불러올 데이터 파일의 이름**

3. 데이터 시각화를 위한 라이브러리 - Matplotlib



데이터를 그래프나 차트 등의 **시각적 형태로 표현하여 이해하고 분석하기 쉽게 만드는 과정**

- 정보 전달의 효율
- 패턴, 상관관계 인식
- 비교와 대조 용이

다양한 데이터 시각화 라이브러리 중 가장 많이 사용되는 라이브러리의 일부

- 간단한 선 그래프
- 산점도
- 히스토그램
- 그래프의 모든 측면을 세밀하게 제어 가능



Matplotlib 라이브러리 import 하기

머신러닝 I

02 머신러닝을 준비하기
위한 라이브러리

3. 데이터 시각화를 위한
라이브러리 -
Matplotlib

```
# import 할 때는 주로 `plt`를 사용  
import matplotlib.pyplot as plt
```

- Matplotlib 내의 함수를 사용하기 위해서는 **Matplotlib.pyplot**을 import
- 다양한 그래프를 그리는 함수부터 그래프 관련 제어 함수 사용 가능



```
plt.plot(["Seoul", "Paris", "Seattle"], [30, 25, 55])  
plt.xlabel('City')  
plt.ylabel('Response')  
plt.title('Experiment Result')  
plt.show()
```

- **plot(x, y, fmt)**

- 입력하는 데이터의 포인트들을 연결하여 **직선 그래프**를 출력하는 함수
- x = x축에 해당하는 데이터 값의 배열 또는 리스트
- y = y축에 해당하는 데이터 값의 배열 또는 리스트
- fmt = 선의 색, 스타일 등을 지정하는 문자열, 생략 가능



```
plt.plot(["Seoul", "Paris", "Seattle"], [30, 25, 55])  
plt.xlabel('City')  
plt.ylabel('Response')  
plt.title('Experiment Result')  
plt.show()
```

- **xlabel(xlabel)**

- xlabel = x축에 표시할 라벨의 텍스트

- **ylabel(ylabel)**

- ylabel = y축에 표시할 라벨의 텍스트



```
plt.plot(["Seoul", "Paris", "Seattle"], [30, 25, 55])  
plt.xlabel('City')  
plt.ylabel('Response')  
plt.title('Experiment Result')  
plt.show()
```

- **title(title)**

- title = 그래프에 표시할 제목

- **show()**

- 그래프를 화면에 표시하기 위해 반드시 호출되어야 하는 함수
- jupyter notebook과 같은 환경에서는 않아도 출력 가능하나, python idle 환경에선 필수

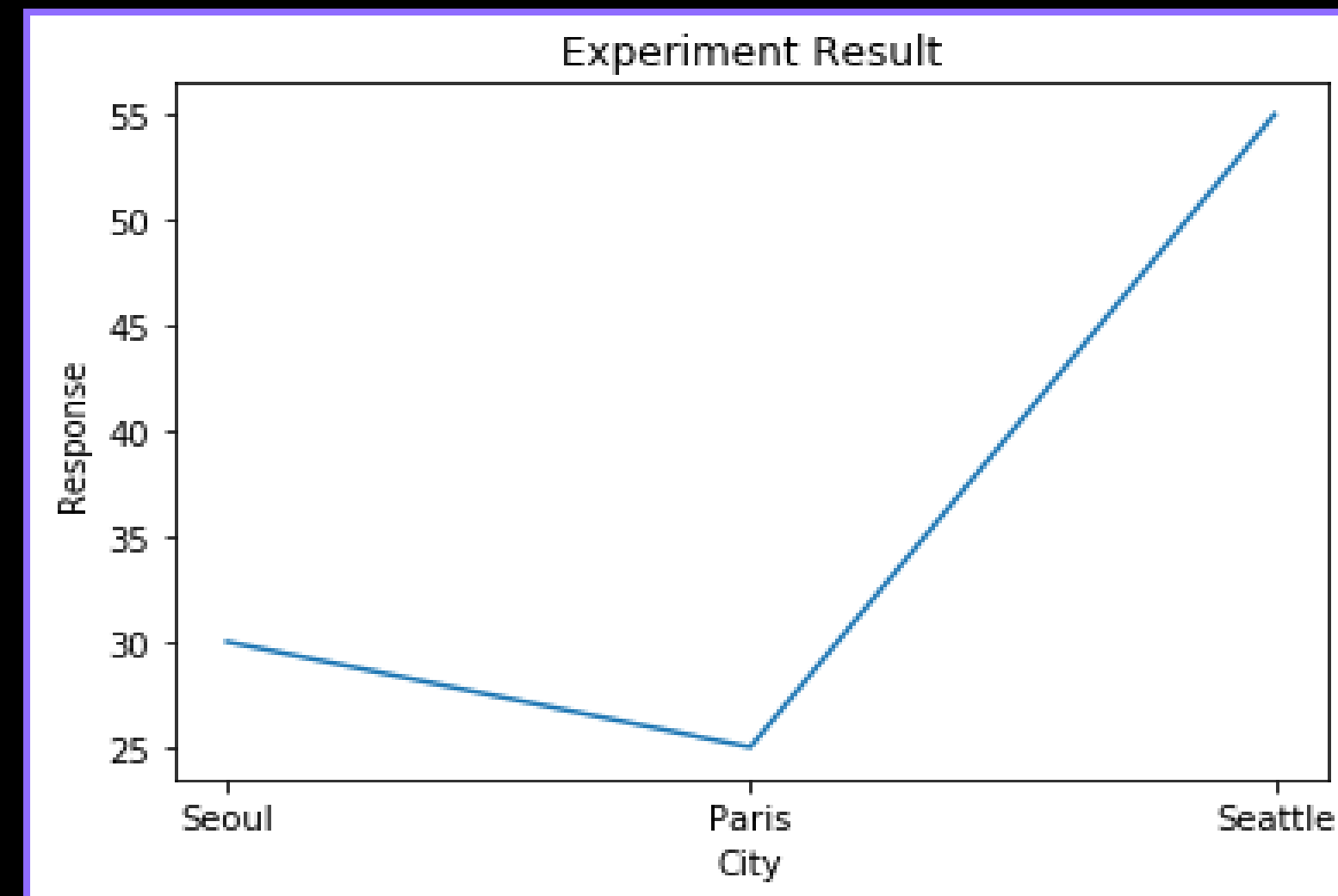


Matplotlib 기본 그래프 그리기

머신러닝 I

02 머신러닝을 준비하기
위한 라이브러리

3. 데이터 시각화를 위한
라이브러리 -
Matplotlib



- 문자열(string)로 입력 받은 경우, 순서에 따라 축에 입력
- (x축 데이터, y축 데이터)인 각 포인트들을 연결한 직선 그래프를 출력


```
plt.scatter([1, 3, 2], [0, -1, 2])  
plt.show()
```

- **scatter(x, y, s, c, marker)**

- 연결되지 않은 point들을 출력하고 싶을 때 사용
- x = x축에 해당하는 데이터 값의 배열 또는 리스트
- y = y축에 해당하는 데이터 값의 배열 또는 리스트
- s = 점의 크기를 지정하는 스칼라, 배열
- c = 점의 색상을 지정하는 스칼라, 배열, 또는 컬러맵 (cmap)
- marker = point의 모양

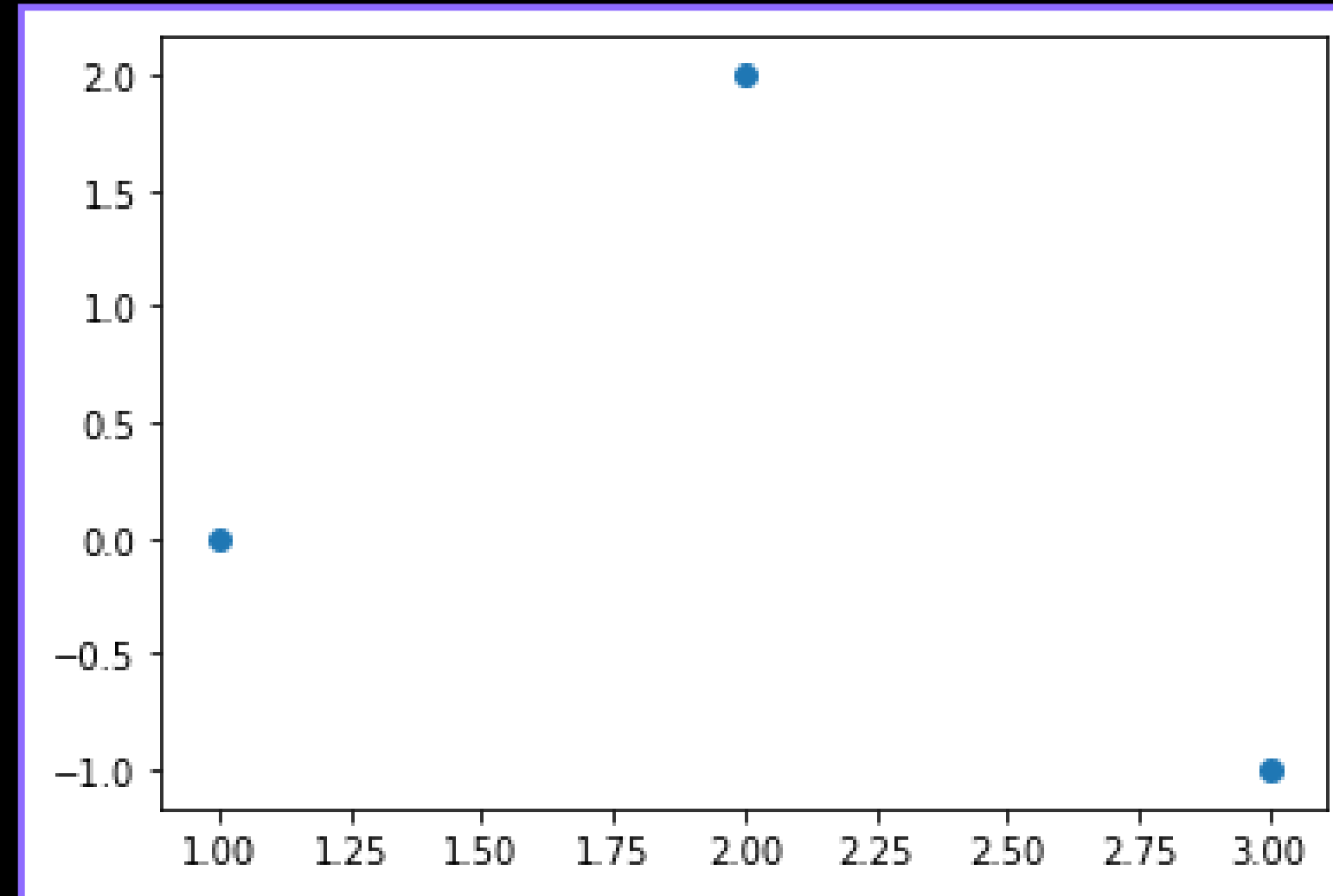
Matplotlib 산점도 그래프 – scatter()

머신러닝 I

02 머신러닝을 준비하기
위한 라이브러리

3. 데이터 시각화를 위한
라이브러리 –
Matplotlib

- cmap
 - 'viridis'
 - 'plasma'
 - 'Blues'
- marker
 - '.' - 작은 점
 - 'o' - 원
 - 'v' - 아래쪽 삼각형
 - 's' - 사각형
 - 'x' - x모양





Matplotlib 히스토그램 그래프- hist()

머신러닝 I

02 머신러닝을 준비하기
위한 라이브러리

3. 데이터 시각화를 위한
라이브러리 -
Matplotlib

```
plt.hist([1, 3, 2, 2, 3, 3, 3, 1, 4, 8, 2, 6, 6], bins=None)  
plt.show()
```

- hist(x, bins)
 - 히스토그램을 출력하는 함수
 - x = 히스토그램을 그릴 데이터의 배열 또는 리스트
 - bins = 데이터를 나눌 구간의 개수, 구간의 경계값 리스트 (default=10)

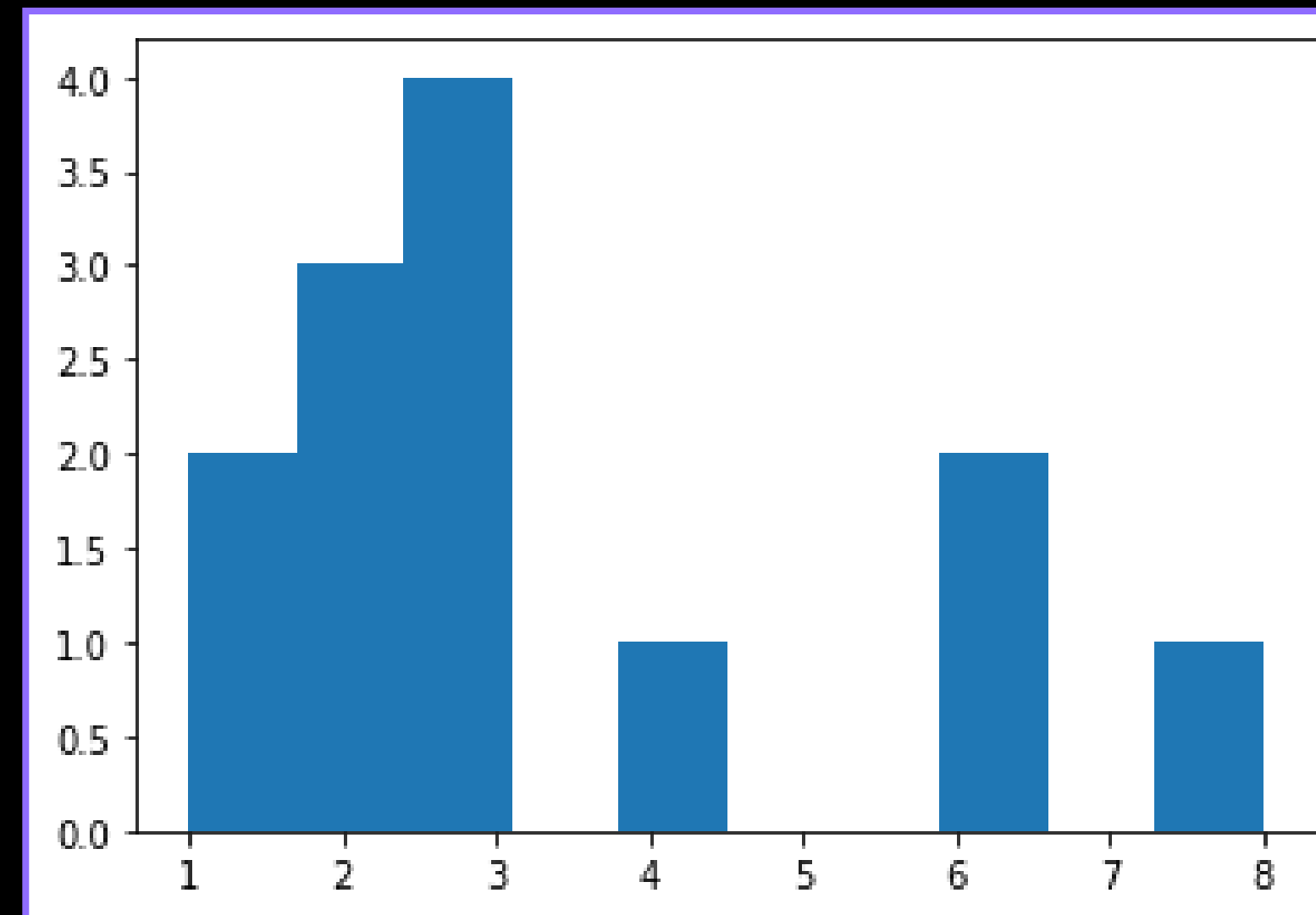


Matplotlib 히스토그램 그래프- hist()

머신러닝 I

02 머신러닝을 준비하기
위한 라이브러리

3. 데이터 시각화를 위한
라이브러리 -
Matplotlib



- 각 리스트 내 값들의 빈도수를 계산하여 나온 출력 결과
- bins 값을 None으로 지정할 시, 자동으로 적절한 구간 개수를 설정하여 출력



Matplotlib 히스토그램 그래프- hist()

머신러닝 I

02 머신러닝을 준비하기
위한 라이브러리

3. 데이터 시각화를 위한
라이브러리 -
Matplotlib

```
plt.hist([1, 3, 2, 2, 3, 3, 3, 1, 4, 8, 2, 6, 6], bins=2)  
plt.show()
```

- bin=2 로 설정 시, 4.5를 기준으로 2개의 range 안에서 개수를 출력
 - 범주를 2개로 나누기 위한 기준 $(1+8)/2=4.5$ 으로 설정

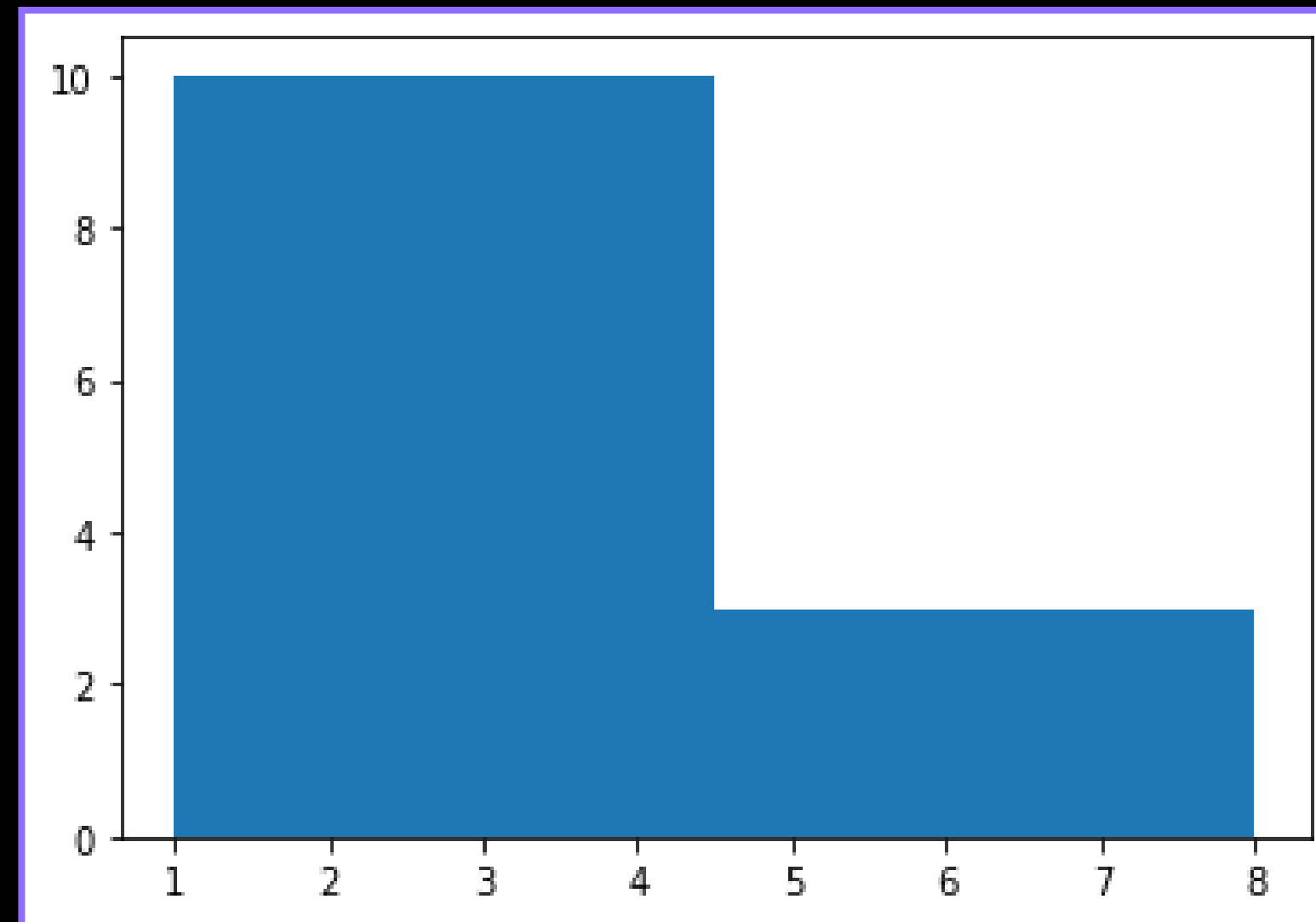


Matplotlib 히스토그램 그래프- hist()

머신러닝 I

02 머신러닝을 준비하기
위한 라이브러리

3. 데이터 시각화를 위한
라이브러리 -
Matplotlib



- bins 값을 2로 지정했을 때의 출력 결과
- 4.5를 기준으로 2개의 막대가 생기는 것을 확인할 수 있음



```
plt.bar(["Seoul", "Paris", "Seattle"], [30, 25, 55])  
plt.xlabel('City')  
plt.ylabel('Response')  
plt.title('Experiment Result')  
plt.show()
```

- **bar(x, height, width)**

- 각 x축 포인트에 따른 y축 값을 막대 그래프로 그리는 함수
- x = 막대 그래프의 x축에 해당하는 위치를 지정하는 배열 또는 리스트
- height = 막대 그래프의 높이를 지정하는 배열 또는 리스트
- width = 막대의 너비를 지정하는 값 (default=0.8)

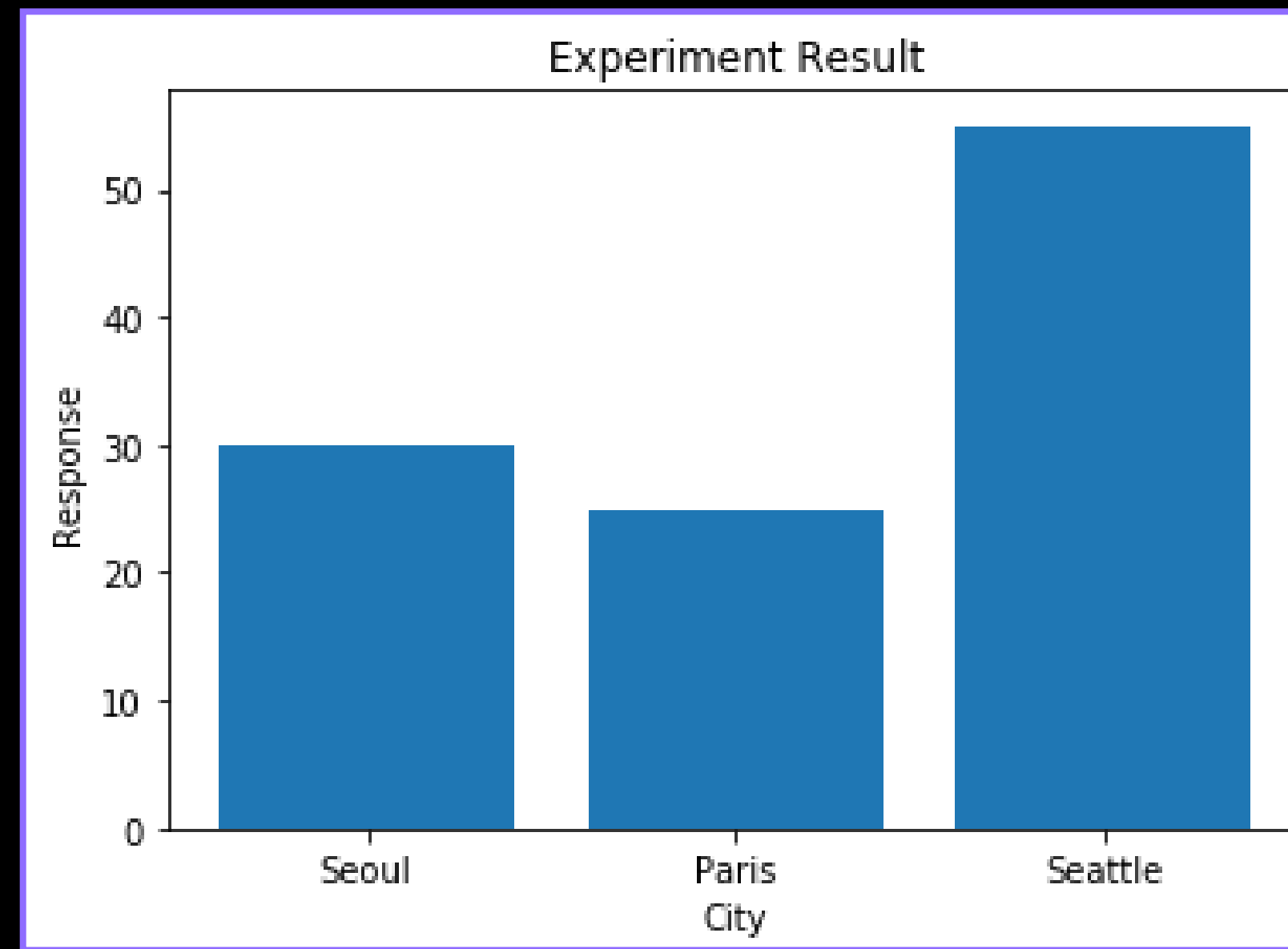


Matplotlib 히스토그램 그래프- bar()

머신러닝 I

02 머신러닝을 준비하기
위한 라이브러리

3. 데이터 시각화를 위한
라이브러리 -
Matplotlib



- 각 카테고리별 값을 막대 그래프로 시각화한 출력 결과
- plot()과 함수를 제외하고 동일한 코드임을 확인

hist()

vs.

bar()

데이터 유형	연속형 데이터 시각화	범주형 데이터 시각화
구간	구간 분할 후 빈도수 계산	카테고리별 분할
x축	데이터 값의 범위	범주형 데이터의 카테고리
y축	각 구간 데이터의 빈도 수	카테고리의 값



```
plt.plot([1, 2, 3], [1, 4, 9])  
plt.plot([2, 3, 4], [5, 6, 7])  
plt.bar([1, 2, 4], [2, 3, 4])  
plt.xlabel('Sequence')  
plt.ylabel('Time(secs)')  
plt.title('Result')  
plt.legend(['Elice', 'AI', 'ML'])  
plt.show()
```

- 여러 개의 그래프를 동시에 출력하기 위해 여러 개의 그래프 함수를 사용
- legend()
 - 범례를 추가해주는 함수

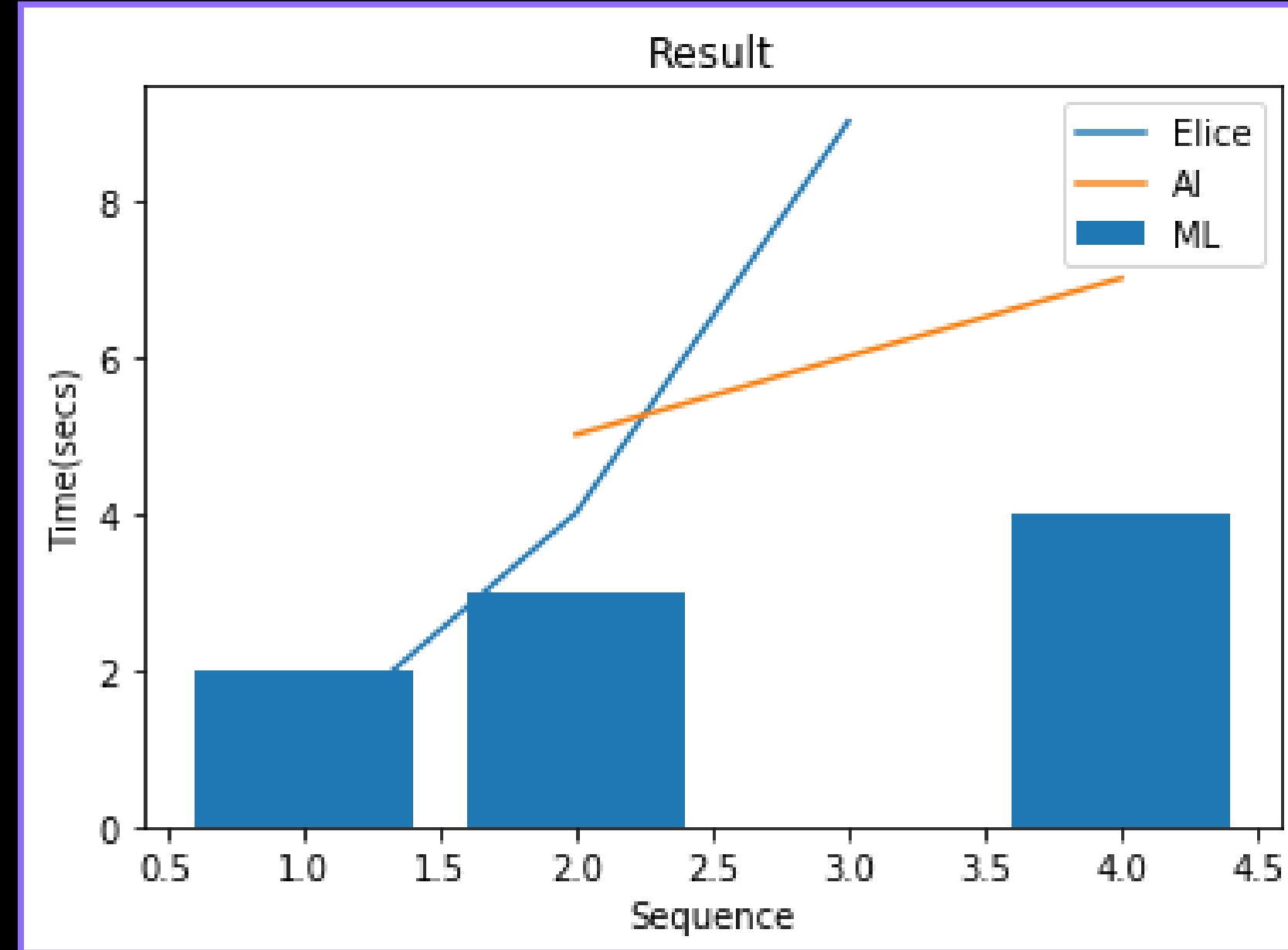


Matplotlib 여러 개의 그래프 출력하기

머신러닝 I

02 머신러닝을 준비하기
위한 라이브러리

3. 데이터 시각화를 위한
라이브러리 -
Matplotlib



- plot 함수 2개, bar 함수 1개를 시각화한 출력 결과

```
x = np.random.rand(5)
y = np.random.rand(5)

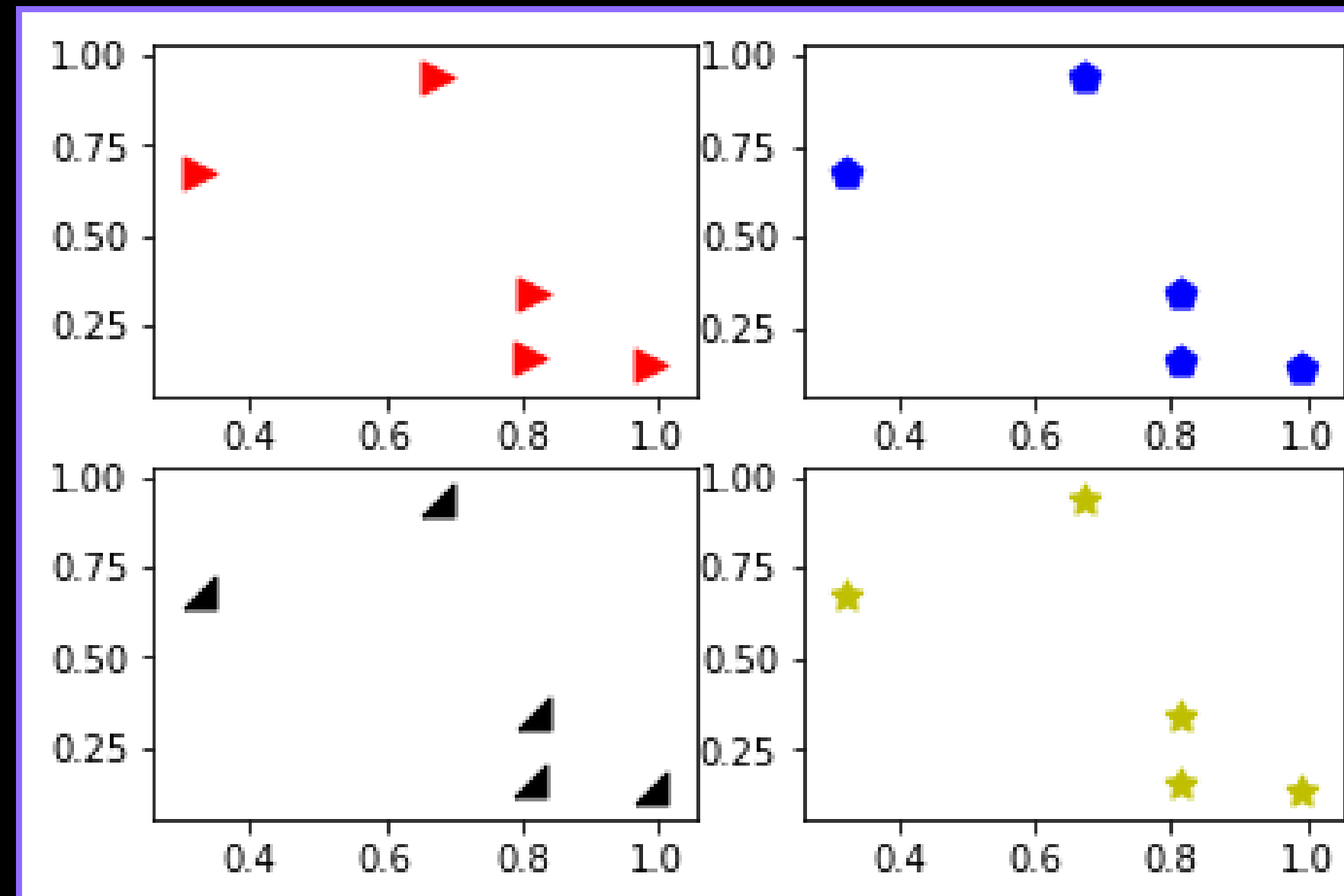
plt.subplot(221)
plt.scatter(x, y, s=80, c='r', marker=">")

plt.subplot(222)
plt.scatter(x, y, s=80, c='b', marker=(5, 0))
...
```

- **subplot(nrows, ncols, index)**
 - 여러 개의 **서브플롯을 생성**하기 위해 사용되는 함수
 - **nrows** = 서브플롯의 **행** 개수
 - **ncols** = 서브플롯의 **열** 개수
 - **index** = 서브플롯의 **인덱스** 지정


```
...  
verts = np.array([[ -1, -1], [ 1, -1], [ 1, 1], [ -1, 1]])  
plt.subplot(223)  
plt.scatter(x, y, s=80, c='k', marker=verts)  
  
plt.subplot(224)  
plt.scatter(x, y, s=80, c='y', marker=(5, 1))  
  
plt.show()
```

- subplot(221)
 - 2, 2 행렬 출력창에서 1번째 출력창으로 설정
- subplot 코드를 그래프 함수에 앞서 입력해야 해당 그래프의 위치 조정 가능



- 2, 2 행렬 모양으로 4개의 산점도를 출력한 결과



```
# a x b 개의 그래프를 그릴 수 있는 figure와 축 설정  
fig, axes = plt.subplots(a, b)
```

- **fig(Figure)**
 - 그림을 나타내는 객체
 - 전체 그림이 존재하는 영역
- **axes**
 - 그림 내의 좌표축을 나타내는 객체
 - 그림 내 여러 개를 가질 수 있음

4. 데이터 시각화를 위한 라이브러리 - Seaborn

Matplotlib을 기반으로 다양한 색상 테마, 통계용 차트 기능이 추가된 패키지

- matplotlib 보다 추가된 기능을 사용하여 데이터를 분석하는 경우
- 세련된 테마로 시각화하는 경우



Seaborn 라이브러리 import 하기

머신러닝 I

02 머신러닝을 준비하기
위한 라이브러리

4. 데이터 시각화를 위한
라이브러리 - Seaborn

```
# import 할 때는 주로 `sns`를 사용  
import seaborn as sns
```

- Seaborn 내 다양한 함수들을 이용하기 위해 seaborn을 import
- 다양한 그래프 그리기 함수 뿐 아니라, 예제 데이터셋을 제공



```
# iris (붓꽃) 데이터 불러오기  
df = sns.load_dataset('iris')
```

- Seaborn의 load_dataset() 함수를 사용하여 데이터를 로드
- 이 외의 데이터를 읽을 시, Pandas 라이브러리의 read_csv(), read_excel과 같은 함수 사용



```
# iris (붓꽃) 데이터 불러오기
```

```
df=sns.load_dataset('iris')
```

```
# 전체 데이터에서 처음 5개의 row 데이터 표시 (내용 확인)
```

```
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

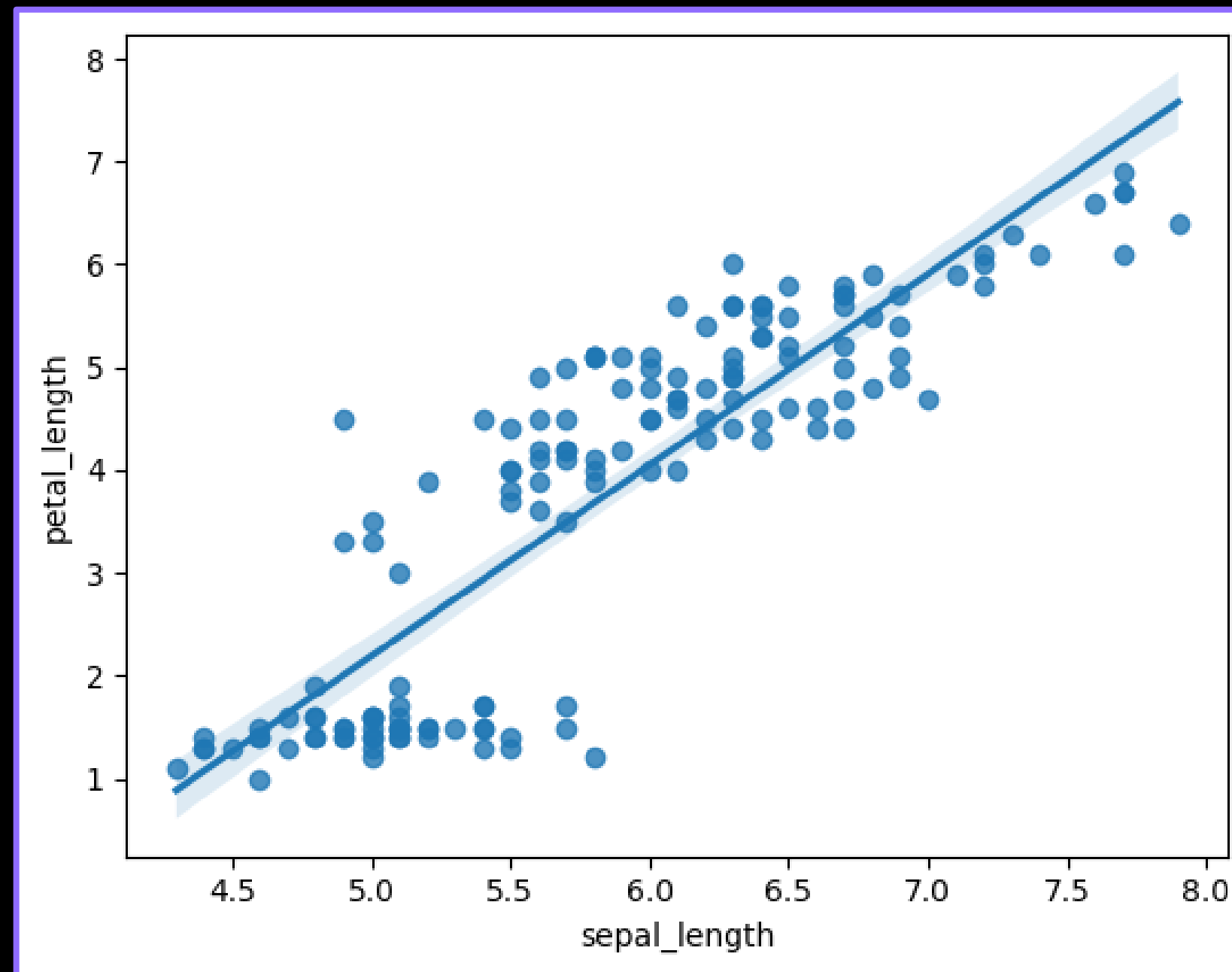


```
df = sns.load_dataset('iris')

sns.regplot(x="sepal_length", y="petal_length", data = df)
plt.show()
```

- **regplot(x, y, data, color)**

- 데이터의 위치인 산점도를 출력할 뿐 아니라, 분포를 근사하는 선도 출력할 수 있는 함수
- x, y = 그래프에 표시될 데이터의 x축, y축
- data = 사용할 데이터프레임
- color = 그래프 색상 설정



- 산점도와 분포를 근사하는 선을 함께 출력한 결과

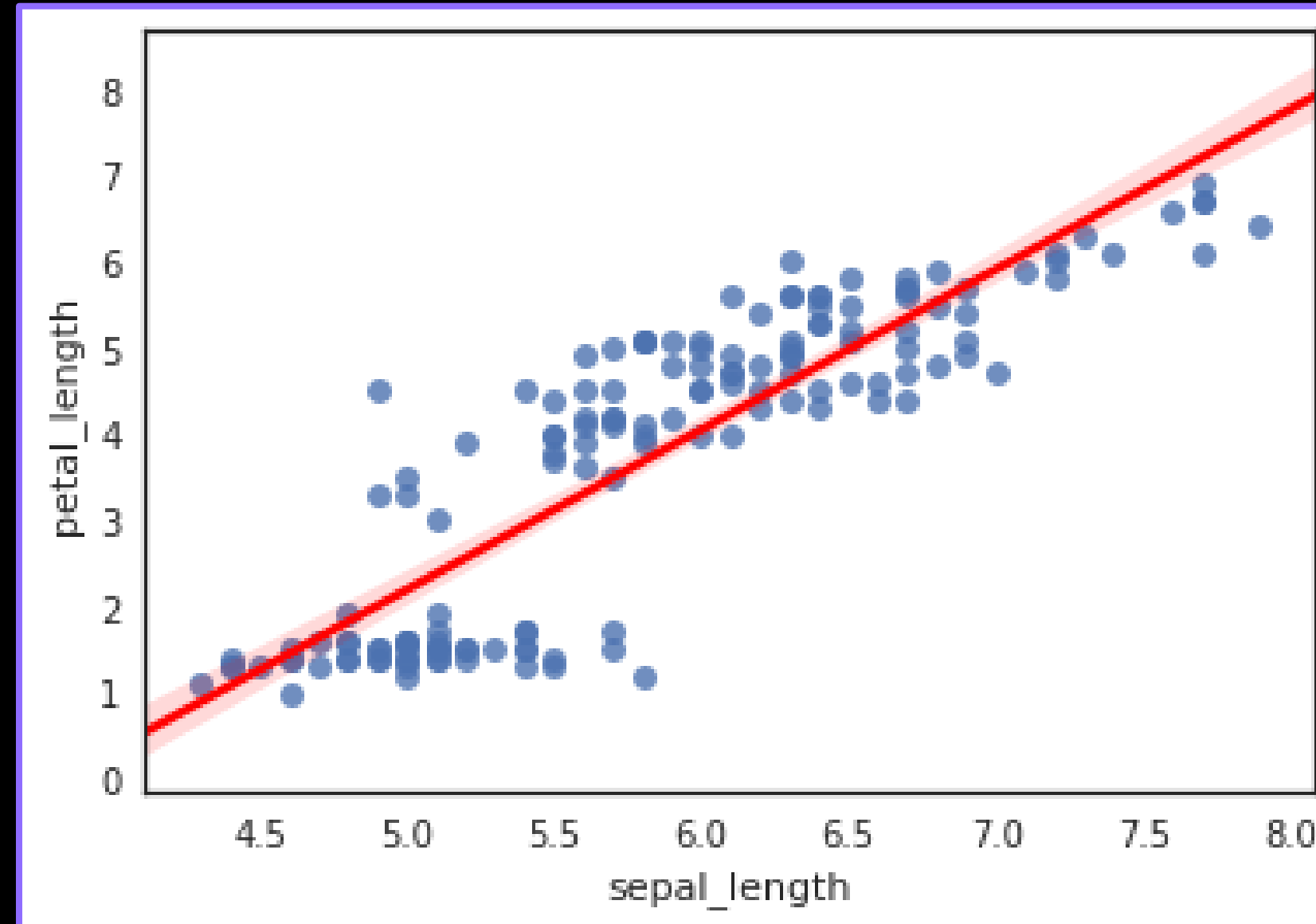


```
df = sns.load_dataset('iris')

sns.regplot(x="sepal_length", y="petal_length", data = df, line_kws={'color': 'red'})
plt.show()
```

- **line_kws**

- 파라미터 설정을 통한 **근사선** 속성 변경
- **color** : 근사선의 색상 지정
- **linewidth** : 근사선의 두께 지정
- **linestyle** : 근사선의 스타일 지정



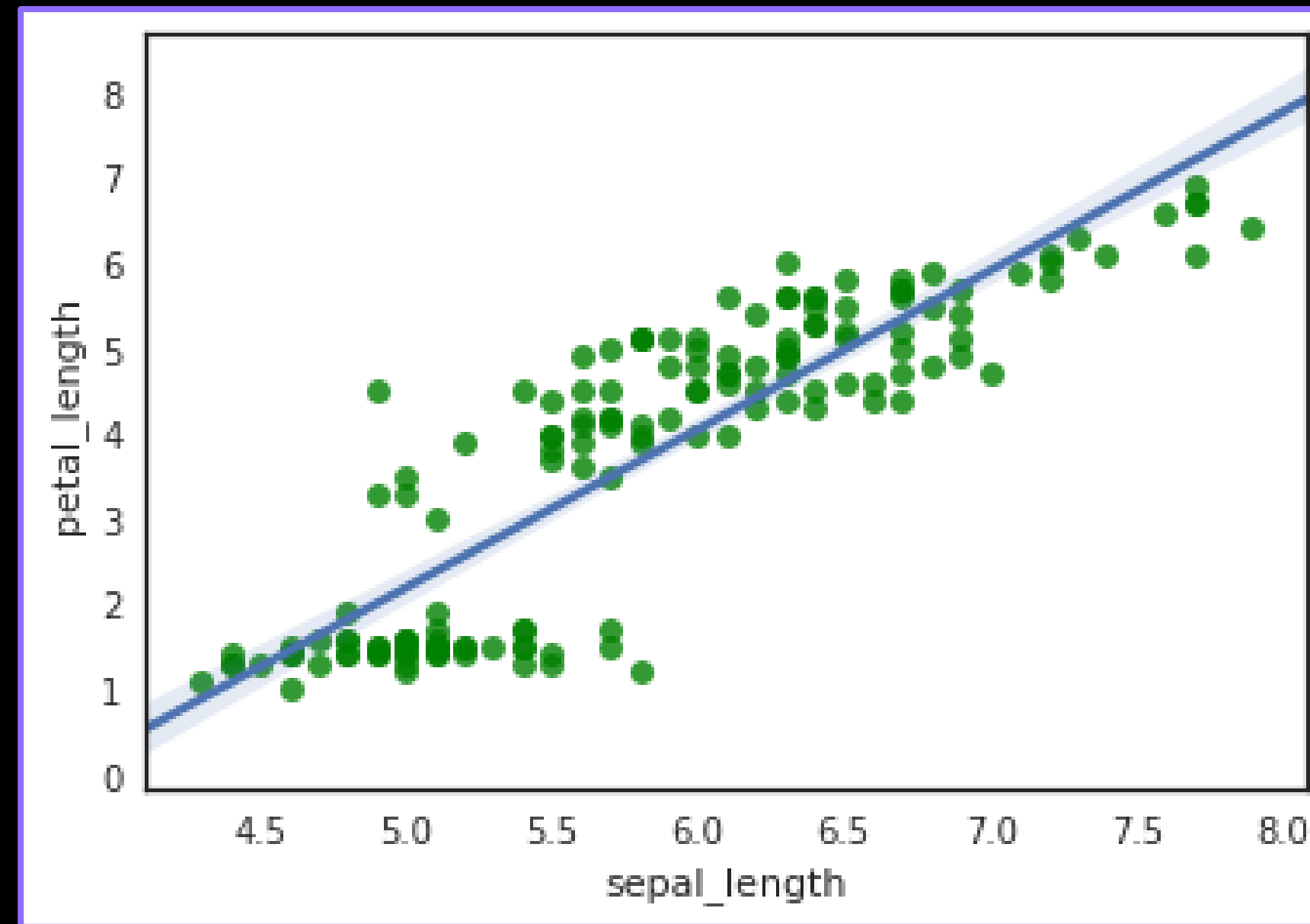
- 근사선의 color 값을 red로 지정한 뒤 출력 결과



```
df = sns.load_dataset('iris')

sns.regplot(x="sepal_length", y="petal_length", data = df, scatter_kws={'color': 'green'})
plt.show()
```

- **scatter_kws**
 - 파라미터 설정을 통한 산점도 속성 변경
 - color : 점의 색상 지정
 - s : 점의 크기를 지정
 - alpha : 점의 투명도 지정



- 산점도의 color 값을 green로 지정한 뒤 출력 결과



```
# 타이타닉 데이터 불러오기
```

```
titanic = sns.load_dataset("titanic")
```

```
# 전체 데이터에서 처음 5개의 row 데이터 표시 (내용 확인)
```

```
titanic.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True



```
sns.countplot(x="class", data=titanic)  
plt.show()
```

- `countplot(x, hue, data)`
 - 범주형 데이터의 빈도를 시각화하는데 사용
 - `x` = 시각화할 데이터 변수
 - `hue` : `x`, `y` 외 구분할 또 다른 변수 지정
 - `data` = 사용할 데이터프레임 지정

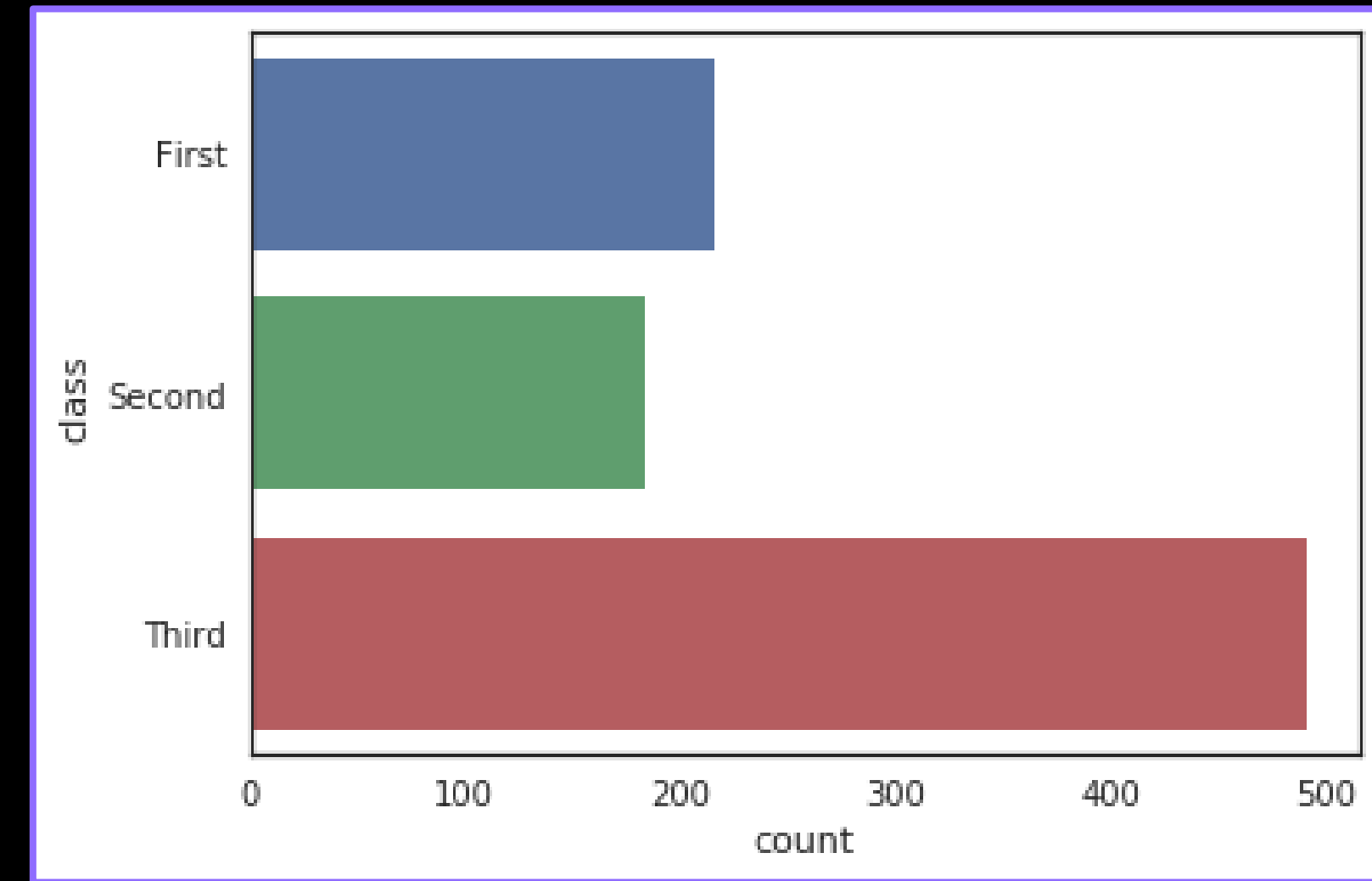
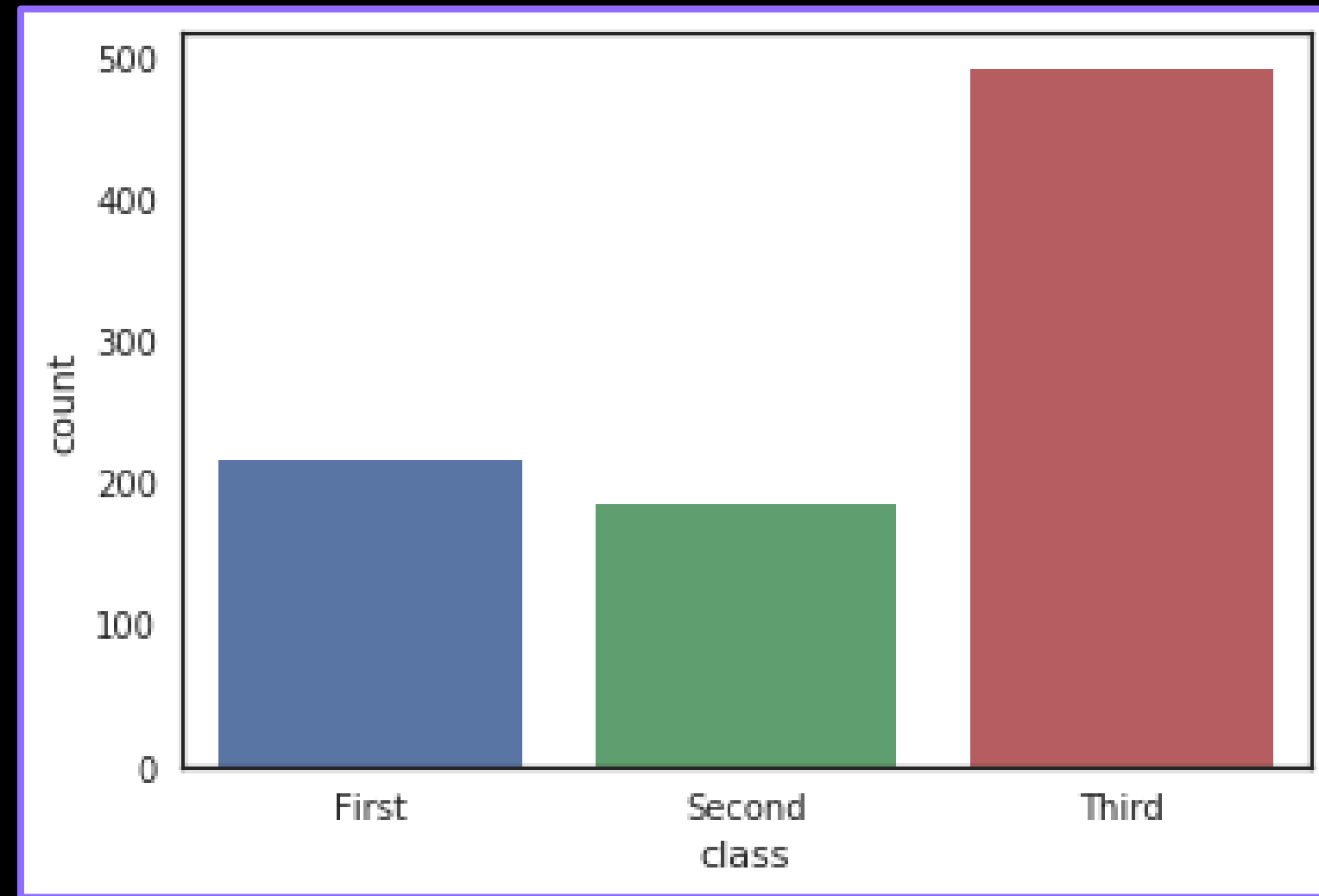


Seaborn 빈도수 막대 그래프

머신러닝 I

02 머신러닝을 준비하기
위한 라이브러리

4. 데이터 시각화를 위한
라이브러리 - Seaborn



- class를 기준으로 countplot을 적용한 출력 결과
- x = "class"가 아닌 y = "class"로 지정 시, y축 기준 countplot을 확인할 수 있음



```
# 팁 데이터 불러오기
```

```
tips = sns.load_dataset('tips')
```

```
# 전체 데이터에서 처음 5개의 row 데이터 표시 (내용 확인)
```

```
tips.head()
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4



```
sns.jointplot(x="total_bill", y="tip", data=tips)  
plt.show()
```

- `jointplot(x, y, data, kind)`
 - 산점도와 히스토그램을 함께 출력
 - `x, y` = 그래프에 표시될 데이터의 x축, y축
 - `data` = 사용할 데이터프레임 지정
 - `kind` = 그래프의 종류 지정 (default=scatter)

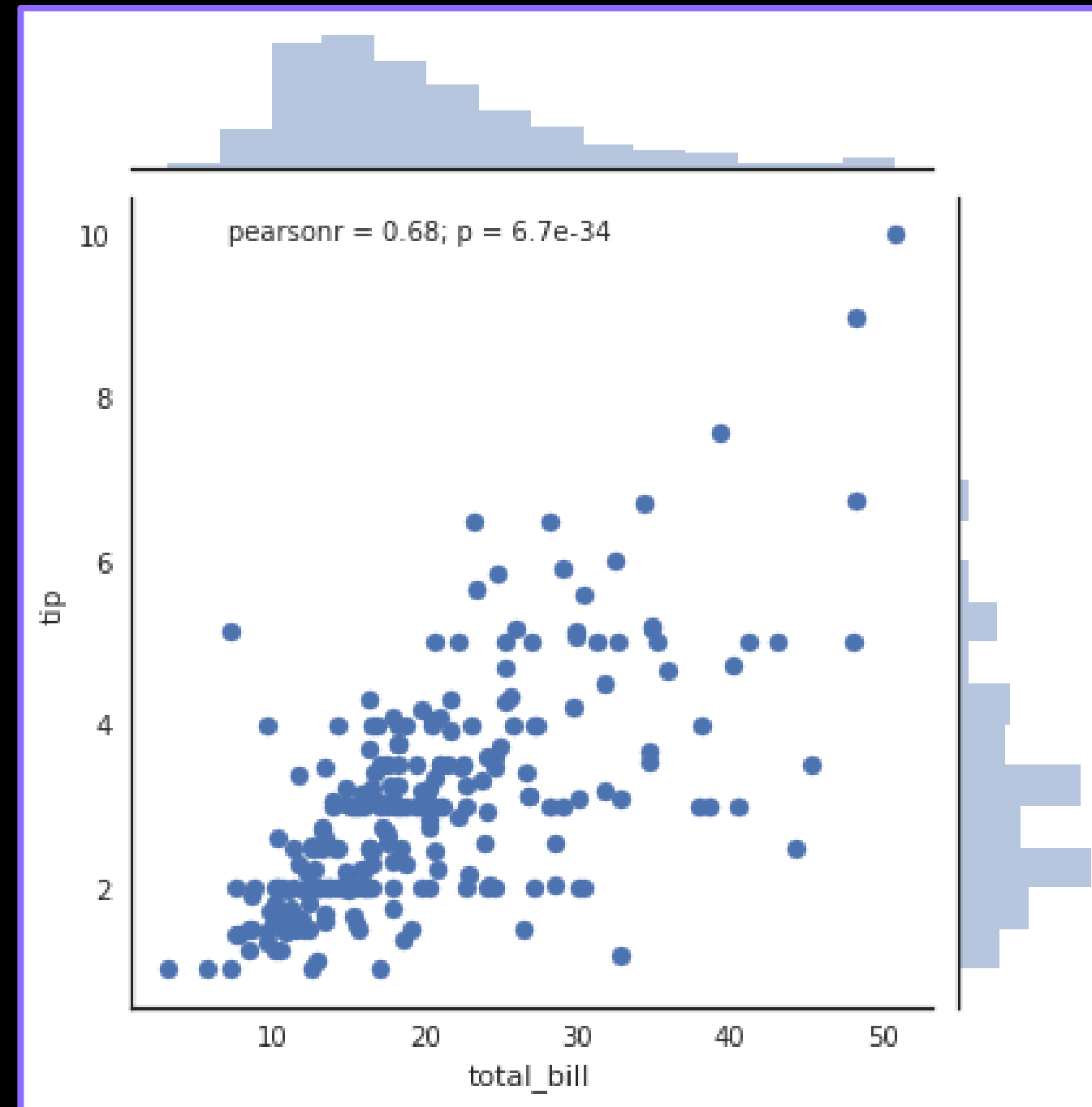


Seaborn 산점도와 히스토그램 그래프

머신러닝 I

02 머신러닝을 준비하기
위한 라이브러리

4. 데이터 시각화를 위한
라이브러리 - Seaborn



- kind를 지정하지 않고 기본값으로 출력한 결과
- 산점도와 히스토그램을 확인할 수 있음



```
# reg (regression + scatter) : 근사선  
sns.jointplot("total_bill", "tip", data=tips, kind="reg")  
plt.show()
```

- kind
 - 'scatter' – 산점도 (default)
 - 'reg' – 근사선 추가
 - 'resid' – 잔차플롯 추가
 - 'kde' – 커널 밀도 추정

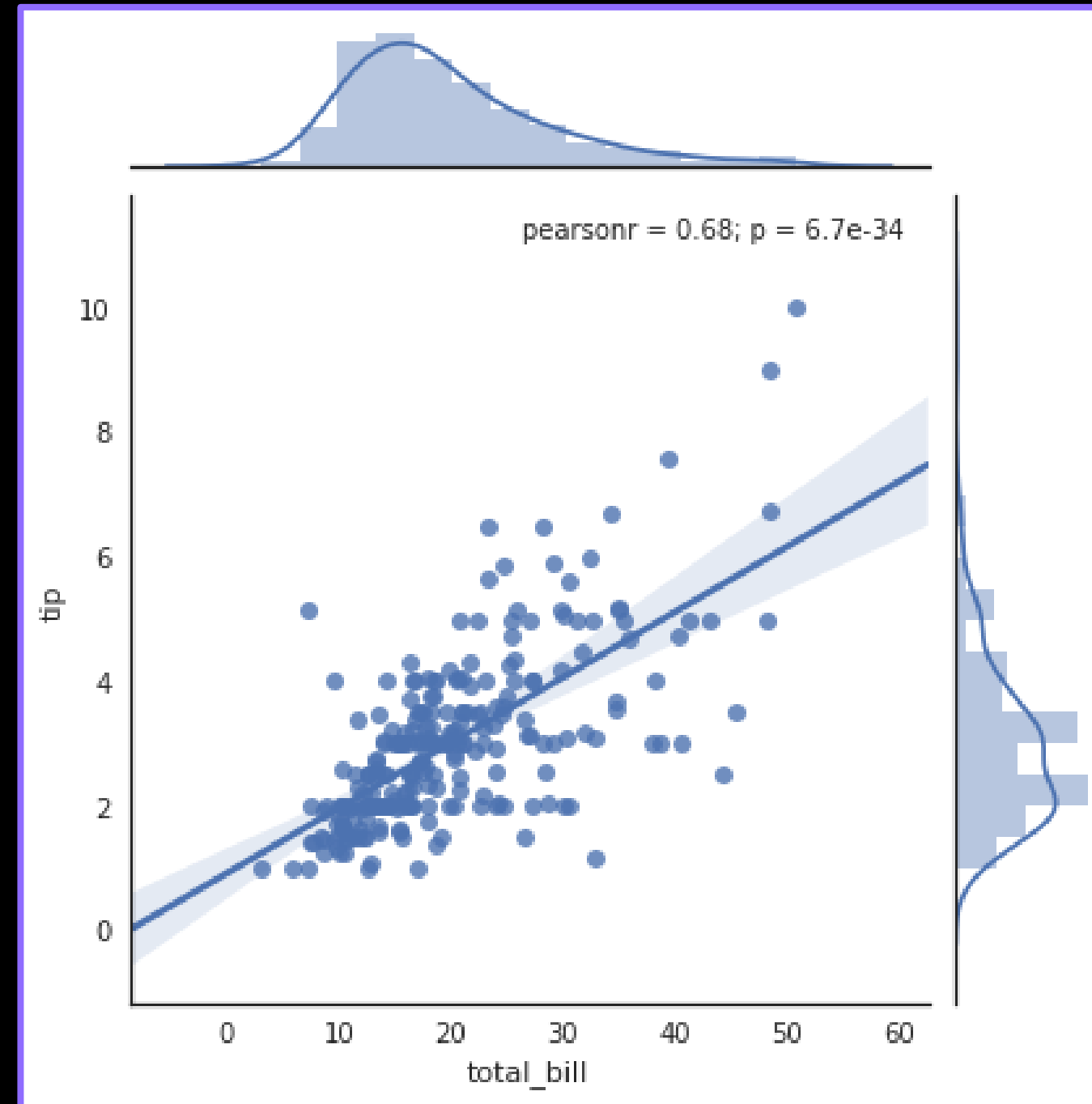


Seaborn 산점도와 히스토그램 그래프

머신러닝 I

02 머신러닝을 준비하기
위한 라이브러리

4. 데이터 시각화를 위한
라이브러리 - Seaborn



- kind에 reg를 지정한 출력 결과
- 근사선이 추가된 산점도와 히스토그램을 확인할 수 있음